SeatSaver

When a payment arrives after the seat is gone, keep the sale instead of losing it.

Aditya Firoda · Razorpay × ISB AI PM Build Challenge · Track 2: Recover and Grow with AI
Live prototype: razorpay-ai-builder.onrender.com
Code and AI build log: github.com/Firoda/RazorPay-AI-Builder

1 The merchant and the problem

SeatSaver is built for small businesses that sell a limited number of seats, such as a pottery studio with eight places in each class. Sometimes a customer's UPI or card payment is confirmed by the bank hours after the app told them it had failed. By then the studio has released the seat, and someone else has booked it.

Razorpay holds that late payment as **"authorised"** for up to three days, which leaves the owner with two bad options. If they do nothing, the money is refunded automatically and they lose a customer who still wanted to come. If they capture it, they have charged for a seat that no longer exists.

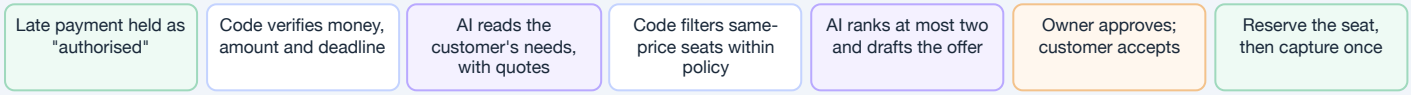
2 Evidence and today's workaround

The mechanism is real and documented: Razorpay describes late authorisation and its three-day capture window, and Xendit documents the same "paid after the order expired" case in Southeast Asia. Public reviews of Razorpay on G2, Capterra, Trustpilot and Reddit show a consistent theme: **exceptions between a payment and the customer's outcome are handled unevenly.**

Today the owner resolves each case by hand, checking the payment dashboard, the booking calendar and WhatsApp, then guessing which alternative the customer might accept, usually after the customer has already complained. Razorpay puts late authorisations at under 0.5% of payments, so this is a focused exception, and its frequency per merchant is the first thing to validate.

3 What I built, for whom, why it matters, and what I left out

SeatSaver is an AI agent that works for the studio owner. When a late payment arrives, it confirms the money is real, reads what the customer actually needs from their own messages ("I can only do Sunday mornings, and I really want Arjun sir"), and proposes up to two sessions at the same price that genuinely fit. The owner approves with one click, the customer picks a session, and SeatSaver **reserves the seat first and captures the payment exactly once**. If nothing fits, it deliberately lets the payment refund and tells the customer why.



Why it matters. The merchant keeps a sale that would otherwise be lost, the customer gets a class they actually want without chasing anyone, and Razorpay turns an awkward automatic refund into a successful payment.

Deliberately left out. Price differences and partial refunds, extra seats, real WhatsApp sending, and owning the merchant's inventory. SeatSaver needs only three calls to the merchant's booking system (check availability, hold a seat, confirm or release), with the owner's own check as a fallback.

4 Key product and AI decisions

- **Code handles every fact; AI only reads language.** Amounts, prices, seat availability, deadlines, reservations and captures are all code. Gemini reads free-text, often Hinglish, messages and ranks options, because that is the part a rule cannot do.
- **The AI can only choose from sessions that code has approved,** so it can never invent availability or offer a different price.
- **The AI never writes amounts or dates.** It writes placeholders, and code fills in the real values.
- **The seat is reserved before the money is captured,** and a ledger of completed actions makes sure a repeated alert or a double click can never charge twice.
- **Review-first by default.** The owner approves every offer, and can later allow automatic sending only for request patterns they trust. Capture always waits for the customer's yes.
- **Customer messages are data, never instructions.** Manipulation attempts go to the owner, and "stop" or "refund" ends the conversation immediately.

5 Success metrics and what I would test next

North Star (L0): Rescued revenue rate. The share of eligible late-payment value that ends as a captured payment for a booking the customer accepted. = ₹ captured after the customer accepts ÷ ₹ of eligible late-authorised payments, measured against a holdout so only incremental rescues count.

It works like a funnel, so the North Star breaks into four **L1 drivers**: Rescued rate = **Coverage** × **Offer rate** × **Acceptance** × **Execution**. Each driver has L2 sub-metrics that show where to act.

L1 · Coverage

Eligible late payments turned into a case before the deadline.

- Detection time after authorisation
- Eligibility accuracy
- Cases needing an integration fix

L1 · Offer rate

Cases where at least one fitting offer reaches the customer.

- Share of cases with a feasible seat
- Owner approval rate, and approval without edits
- Time to the owner's decision

L1 · Acceptance

Offers the customer accepts.

- Offers matching every stated need (AI quality)
- Time from authorisation to offer
- Customer reply rate

L1 · Execution

Accepted offers that end reserved and captured.

- Seats lost before reservation
- Capture failures and retry success
- Bookings confirmed to the customer

Guardrails (must not get worse)

- **Zero** charges without an accepted, reserved seat, and zero double captures
- Offers that contradict the customer's request
- Payments refunded only because nobody acted in time
- Complaints and opt-outs after contact
- Owner minutes per case and share escalated

How it is proven so far

- Development set: **17 of 18** correct; fresh held-out cases: **3 of 3**
- A fixed "next free slot" rule: **0%**, with violations on every run
- **Zero** wrong money actions; 53 automated tests
- A real Razorpay test payment captured end to end
- Synthetic cases show decision quality, not real-world uplift

What I would test next

- **Pilot** with 3 to 5 studios for 4 to 6 weeks, with a randomised holdout on today's process; measure incremental retained revenue per eligible case
- **Frequency and acceptance:** how often this happens per merchant, and how often customers take an alternative
- **Owner trust:** approval-without-edit rate, before enabling auto-send
- **Real WhatsApp** delivery and reply rates

All demo data is synthetic. Payments use Razorpay test mode only; no real money moves.

Tools: Claude Code, Gemini 3.8 Flash, ChatGPT images, Google Flow